Singular Value Decomposition

Boston University CS 506 - Lance Galletti

Warm Up: working with text data

Warm Up: working with text data

Paris

Warm Up: working with text data

Paris France

Warm Up: working with text data

Paris France Rome

Warm Up: working with text data

Paris France Rome Italy

Warm Up: working with text data

Paris France Rome Italy

How do we convert these words to numbers that can be used by algorithms?

Warm Up: working with text data

Paris	France	Rome	Italy
1	2	3	4

Warm Up: working with text data

Paris	France	Rome	Italy
1	2	3	4

Paris and France close ✅
Rome and Italy close ✅
France and Rome equally close to Paris and France ❌

Warm Up: working with text data

France	Paris	Rome	Italy
1	2	3	4

Paris and France close ✅
Rome and Italy close ✅
France and Italy twice as far as Paris and Rome ❌

Warm Up: working with text data

Paris Rome

France Italy

Warm Up: working with text data

Paris Rome

France Italy

Paris and France close ✅
Rome and Italy close ✅
France and Italy twice as far as Paris and Rome ✅
Paris and Italy as far as Rome and France ✅

Warm Up: working with text data

Paris Rome

France Italy

What if we now add Madrid and Spain?

Warm Up: working with text data

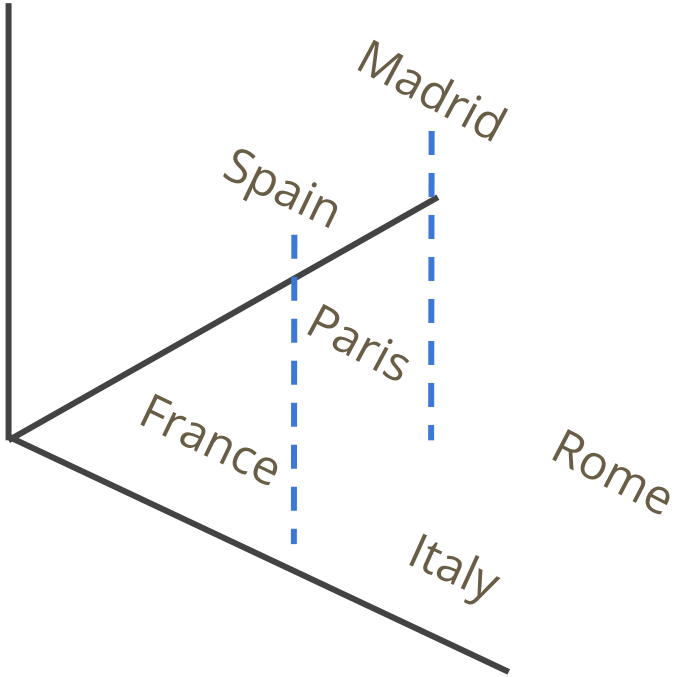
Paris

France

Rome

Italy

Warm Up: working with text data



Warm Up: working with text data

Words -> vectors

How can we represent the following words:

Capital France Paris

Warm Up: working with text data

Capital France Paris

Warm Up: working with text data

Capital	France	Paris
1	0	0
0	1	0
0	0	1

One-hot encoding

Warm Up: working with text data

Capital • France = 0

Paris • France = 0

Capital • Paris = 0

Capital	France	Paris
1	0	0
0	1	0
0	0	1

One-hot encoding

Warm Up: working with text data

But… The capital of France is Paris.

Capital • France = 0

Paris • France = 0

Capital • Paris = 0

Capital	France	Paris
1	0	0
0	1	0
0	0	1

One-hot encoding

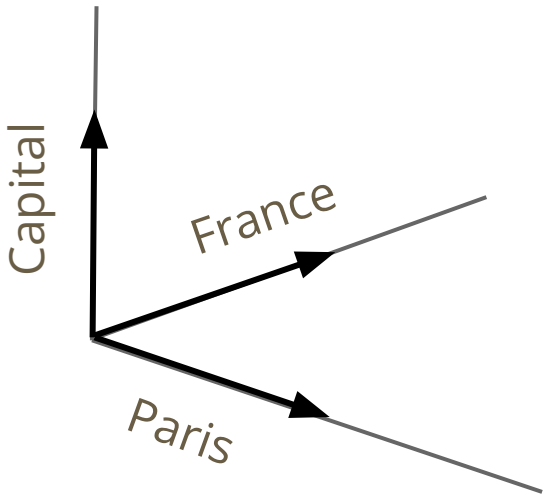
Warm Up: working with text data

But… The capital of France is Paris.

Capital • France = 0

Paris • France = 0

Capital • Paris = 0



Capital	France	Paris
1	0	0
0	1	0
0	0	1

One-hot encoding

Warm Up: working with text data

Capital	France	Paris
1	0	0
0	1	0
0	0	1

One-hot encoding

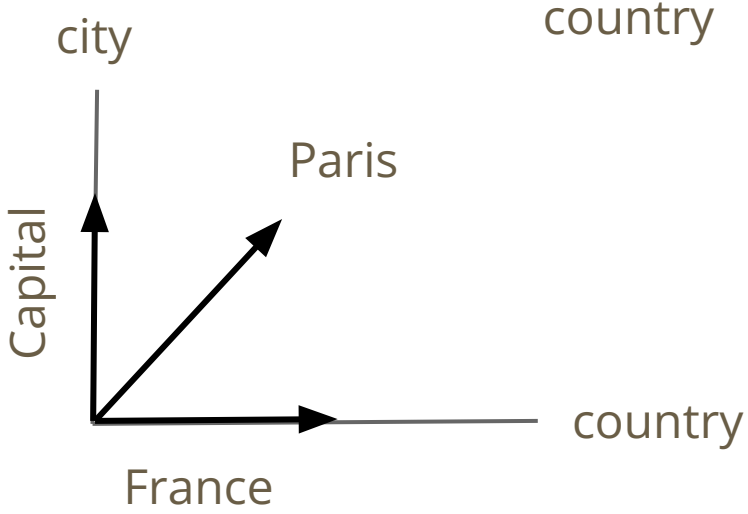
	Capital	France	Paris
city	1	0	1
country	0	1	1

Embedding

Warm Up: working with text data

	Capital	France	Paris
city	1	0	1
country	0	1	1

Embedding



Warm Up: working with text data

	Capital	France	Paris
city	1	0	1
country	0	1	1

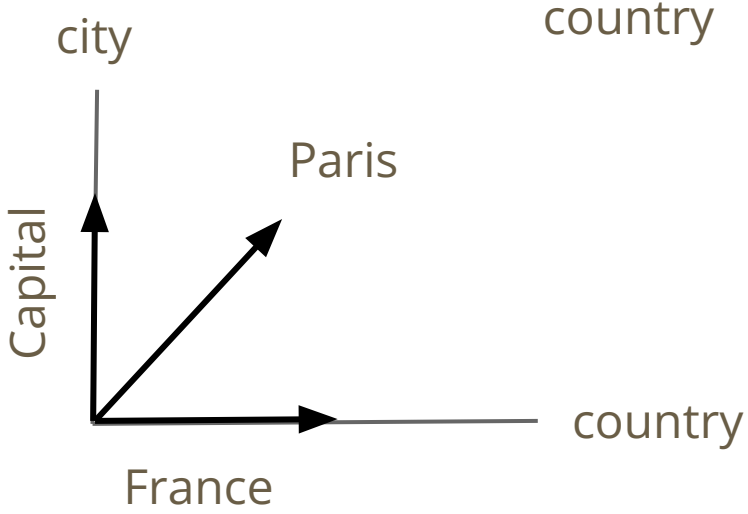
Embedding

Capital + France = Paris

Capital • France = 0

Paris • France = 1

Capital • Paris = 1



Warm Up: working with text data

	Capital	France	Paris
city	1	0	1
country	0	1	1

Embedding

How strongly the word expresses the concept

Warm Up: working with text data

	Capital	France	Paris
city	.8	.1	1
country	.1	1	.7

Embedding

Warm Up: working with text data

Capital + France = [.9, 1.1]

Capital • France = .18

Paris • France = .8

Capital • Paris = 0.87

	Capital	France	Paris
city	.8	.1	1
country	.1	1	.7

Embedding

How do we find this embedding space?

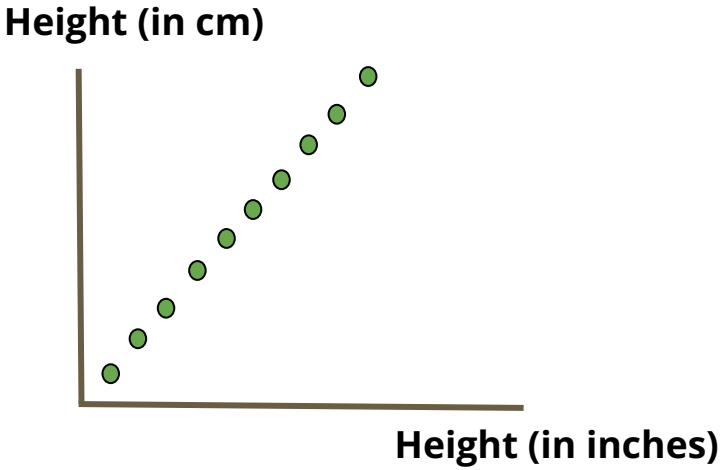
Guiding principle: words that appear together a lot should share semantic meaning.

- the **capital** of **france** is **paris**
- **paris** is a beautiful **city**
- **france** is a **country** in europe
- the **capital** of **italy** is **rome**
- **rome** is an ancient **city**

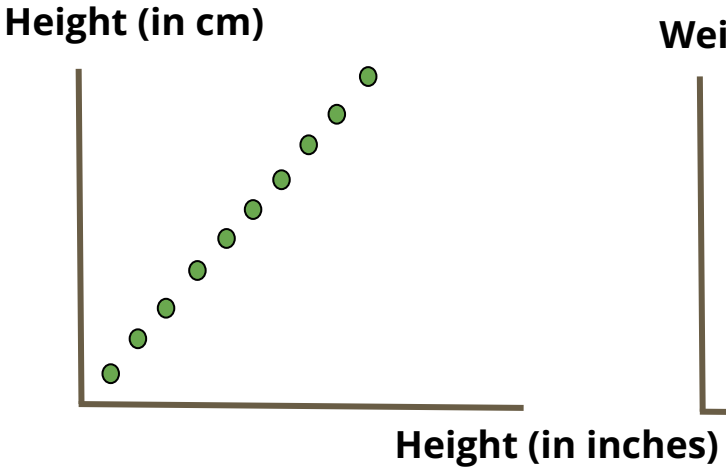
Singular Value Decomposition

Boston University CS 506 - Lance Galletti

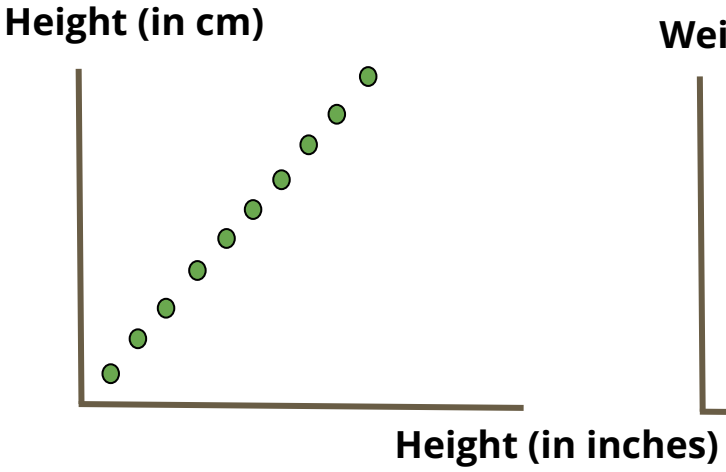
Characteristics of a dataset to look for

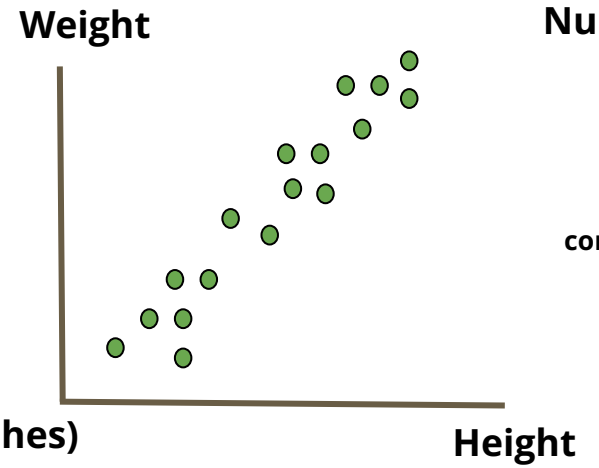


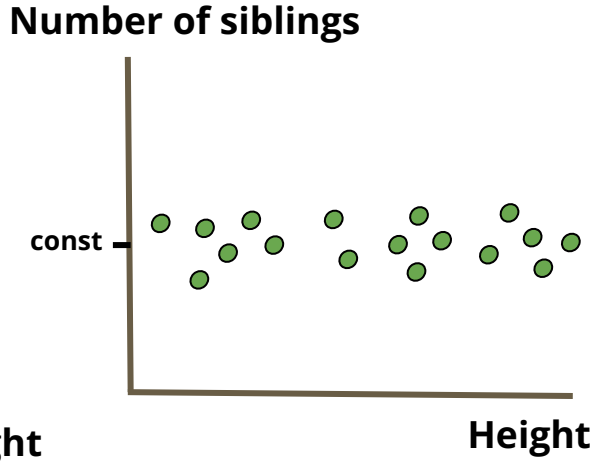
Characteristics of a dataset to look for

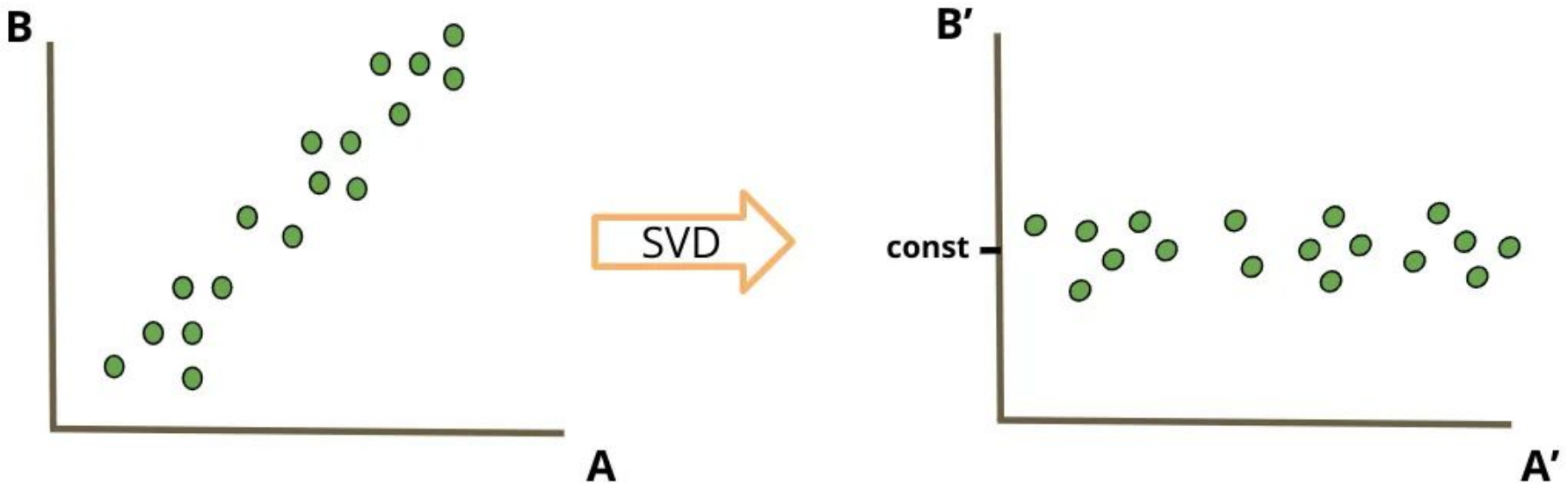


Characteristics of a dataset to look for

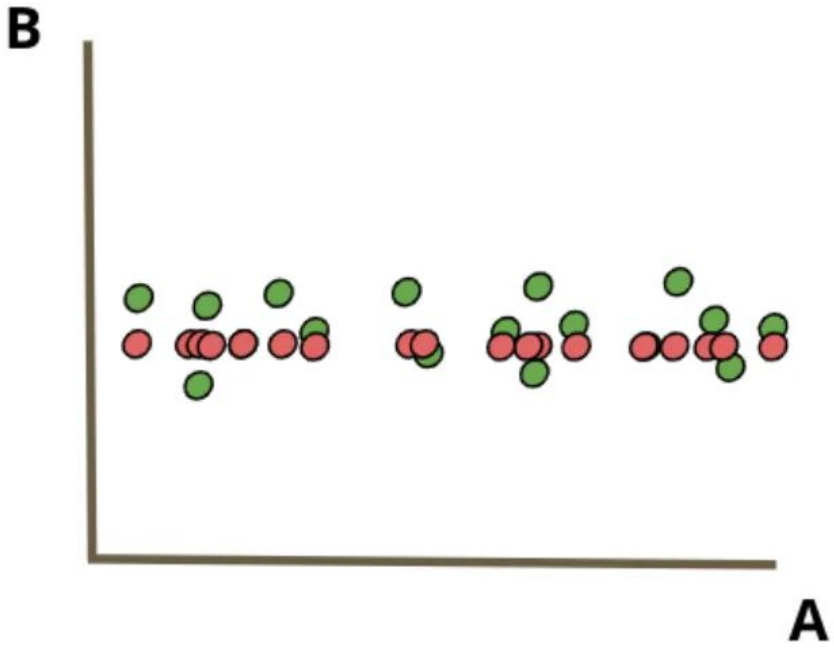


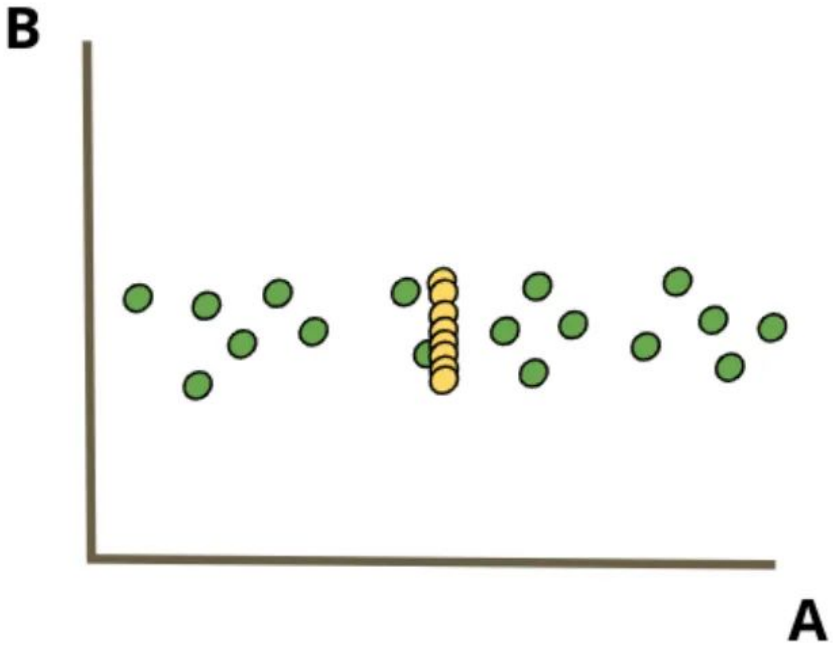




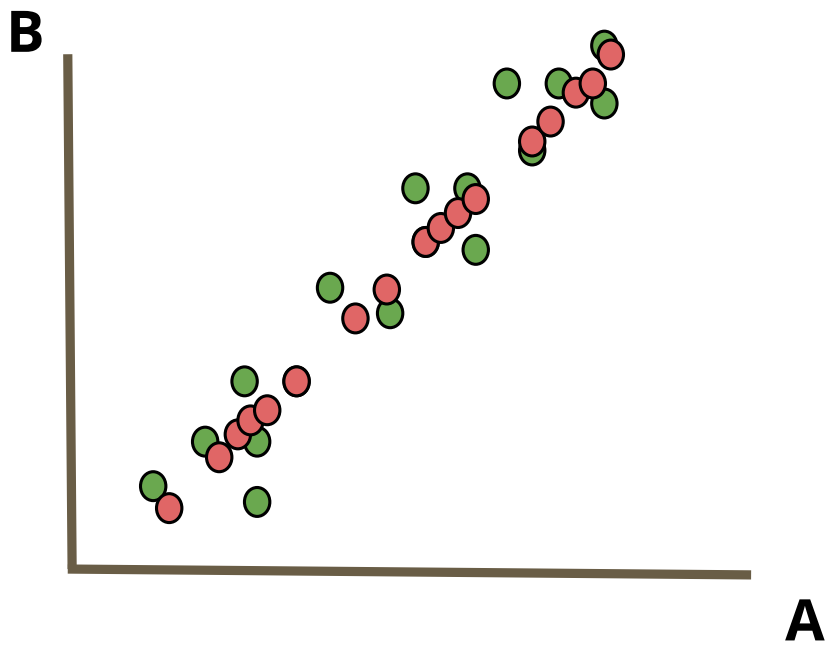

B
A
SVD
B'
const
A'

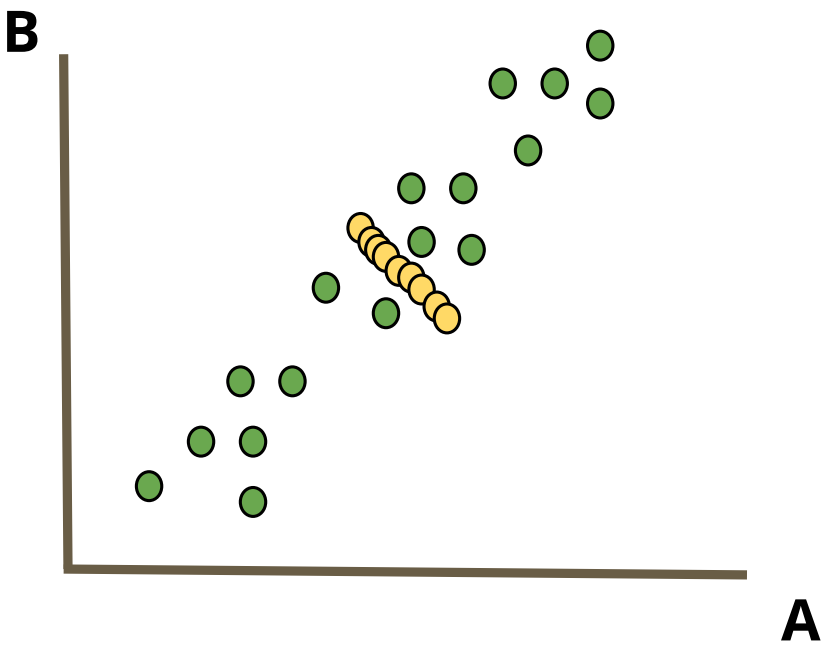
Dimension reduction / Feature Selection





Data Approximation / Denoising





$$
\left.\begin{array}{c} \\ \\ \\ \\ \\ \end{array}\right\}\text{\textbf{n} data points}
\underbrace{\begin{pmatrix} x_{11} & \cdots & x_{1j} & \cdots & x_{1m} \\ \vdots & \ddots & \vdots & & \vdots \\ x_{i1} & \cdots & x_{ij} & \cdots & x_{im} \\ \vdots & & \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nj} & \cdots & x_{nm} \end{pmatrix}}_{\text{\textbf{m} features}}
$$

Goal

Examine this matrix and uncover its linear algebraic properties to:

1. Approximate A with a smaller matrix B that is easier to store but contains similar information as A (remove noise)

$$
\begin{array}{cc}
 & \begin{array}{ccccc} \textbf{A} & \textbf{...} & \textbf{J} & \textbf{...} & \textbf{M} \end{array} \\
\textbf{n} \text{ data points} \left\{ \right. & \underbrace{\begin{pmatrix} x_{11} & \cdots & x_{1j} & \cdots & x_{1m} \\ \vdots & \ddots & \vdots & & \vdots \\ x_{i1} & \cdots & x_{ij} & \cdots & x_{im} \\ \vdots & & \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nj} & \cdots & x_{nm} \end{pmatrix}}_{\textbf{m} \text{ features}}
\end{array}
\qquad
\begin{array}{c}
\begin{array}{ccccc} \textbf{A'} & \textbf{...} & \textbf{J'} & \textbf{...} & \textbf{M'} \end{array} \\
\underbrace{\begin{pmatrix} x_{11} & \cdots & x_{1j} & \cdots & x_{1m} \\ \vdots & \ddots & \vdots & & \vdots \\ x_{i1} & \cdots & x_{ij} & \cdots & x_{im} \\ \vdots & & \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nj} & \cdots & x_{nm} \end{pmatrix}}_{\textbf{m} \text{ features}}
\end{array}
$$

Goal

Examine this matrix and uncover its linear algebraic properties to:

1. Approximate A with a smaller matrix B that is easier to store but contains similar information as A (remove noise)
2. Dimensionality Reduction / Feature Extraction

$$
\begin{array}{cc}
 & \begin{array}{ccccc} \mathbf{A} & \mathbf{\ldots} & \mathbf{J} & \mathbf{\ldots} & \mathbf{M} \end{array} \\
\begin{array}{c} \\ \\ \textbf{n} \text{ data} \\ \text{points} \\ \\ \\ \end{array} &
\left\{ \underbrace{\begin{pmatrix}
x_{11} & \cdots & x_{1j} & \cdots & x_{1m} \\
\vdots & \ddots & \vdots & & \vdots \\
x_{i1} & \cdots & x_{ij} & \cdots & x_{im} \\
\vdots & & \vdots & \ddots & \vdots \\
x_{n1} & \cdots & x_{nj} & \cdots & x_{nm}
\end{pmatrix}}_{\textbf{m} \text{ features}} \right.
\end{array}
$$

$$
\begin{array}{c}
\begin{array}{ccc} \mathbf{A''} & \mathbf{\ldots} & \mathbf{J''} \end{array} \\
\underbrace{\begin{pmatrix}
x_{11} & \cdots & x_{1j} \\
\vdots & \ddots & \vdots \\
x_{i1} & \cdots & x_{ij} \\
\vdots & & \vdots \\
x_{n1} & \cdots & x_{nj}
\end{pmatrix}}_{\textbf{j} \text{ features}}
\end{array}
$$

Goal

Examine this matrix and uncover its linear algebraic properties to:

1. Approximate A with a smaller matrix B that is easier to store but contains similar information as A (remove noise)
2. Dimensionality Reduction / Feature Extraction
3. Anomaly Detection & Denoising

$$
\begin{array}{c}
\textbf{n} \text{ data points} \\
\left\{ \begin{array}{c}
\begin{pmatrix}
x_{11} & \cdots & x_{1j} & \cdots & x_{1m} \\
\vdots & \ddots & \vdots & & \vdots \\
\boxed{x_{i1} \quad \cdots \quad x_{ij} \quad \cdots \quad x_{im}} \\
\vdots & & \vdots & \ddots & \vdots \\
x_{n1} & \cdots & x_{nj} & \cdots & x_{nm}
\end{pmatrix} \\
\underbrace{\qquad\qquad\qquad\qquad\qquad\qquad}_{\textbf{m} \text{ features}}
\end{array} \right.
\end{array}
$$

Linear Algebra Review

Definition: The vectors in a set $\mathbf{V = \{ \vec{v}_1, \ldots, \vec{v}_n \}}$ are **linearly independent** if

$$\mathbf{a_1\vec{v}_1 + \ldots + a_n\vec{v}_n = \vec{0}}$$

can only be satisfied by $\mathbf{a_i = 0}$

Note: this means no vector in that set can be expressed as a **linear combination** of other vectors in the set.

Linear Algebra Review

Definition:

The **determinant** of a square matrix A is a scalar value that encodes properties about the **linear mapping** described by A.

2x2:

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

det(A) = ad - bc

Linear Algebra Review

Definition:

The **determinant** of a square matrix A is a scalar value that encodes properties about the **linear mapping** described by A.

3x3:

$$A = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \quad det(A) = a \cdot det\left(\begin{smallmatrix} e & f \\ h & i \end{smallmatrix}\right) - b \cdot det\left(\begin{smallmatrix} d & f \\ g & i \end{smallmatrix}\right) + c \cdot det\left(\begin{smallmatrix} d & e \\ g & h \end{smallmatrix}\right)$$

Linear Algebra Review

Definition:

The **determinant** of a square matrix A is a scalar value that encodes properties about the **linear mapping** described by A.

n X n:

Can recursively compute it. How?

Linear Algebra Review

Property:

n vectors **{** $\vec{v}_1, \ldots, \vec{v}_n$ **}** in an n-dimensional space are **linearly independent** iff the matrix **A**:

$$\mathbf{A} = [\,\vec{v}_1, \ldots, \vec{v}_n\,] \quad (n \times n)$$

has non-zero determinant.

Q: Can **m > n** vectors in an **n**-dimensional space be linearly independent?

Linear Algebra Review

Definition:

The **rank** of a matrix **A** is the dimension of the vector space spanned by its column space. This is equivalent to the maximal number of linearly independent columns / rows of **A**.

Definition:

A matrix **A** is **full-rank** iff **rank(A) = min(m, n)**

Note: Get the rank of a matrix through the **Gram-Schmidt process**

Matrix Factorization

Any matrix **A** of rank **k** can be factored as

$$\mathbf{A} = \mathbf{UV}$$

where

U is **n x k**
V is **k x m**

Matrix Factorization

To store an **n x m** matrix **A** requires storing **m · n** values.

However, if the rank of the matrix of **A** is **k**, since **A** can be factored as

$$\mathbf{A} = \mathbf{UV}$$

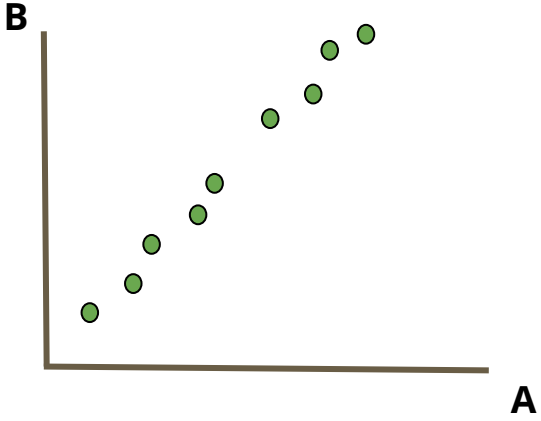
which requires storing **k(m + n)** values.

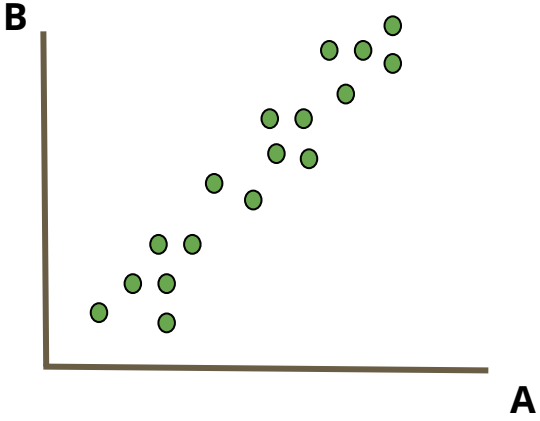
In Practice

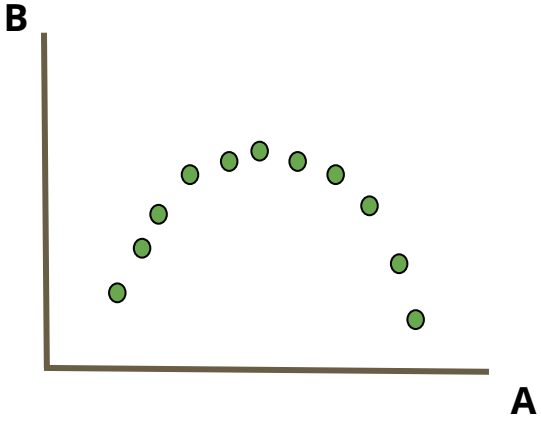
Most datasets are full rank despite containing a lot of redundant /similar information…

In Practice

Most datasets are full rank despite containing a lot of redundant /similar information...







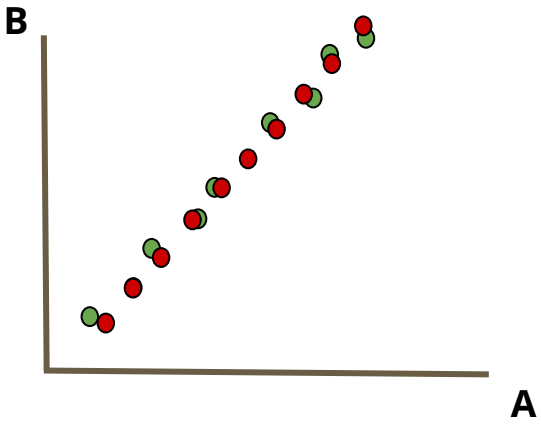
In Practice

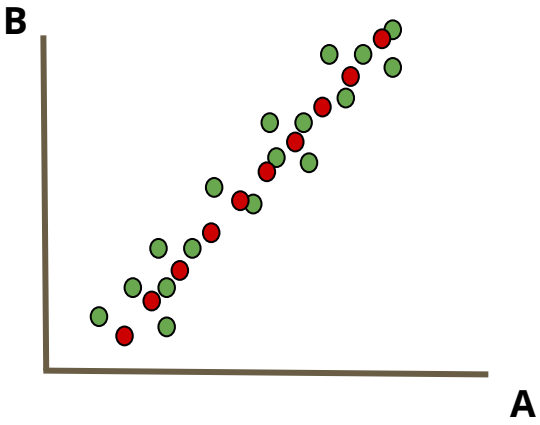
Most datasets are full rank despite containing a lot of redundant /similar information...

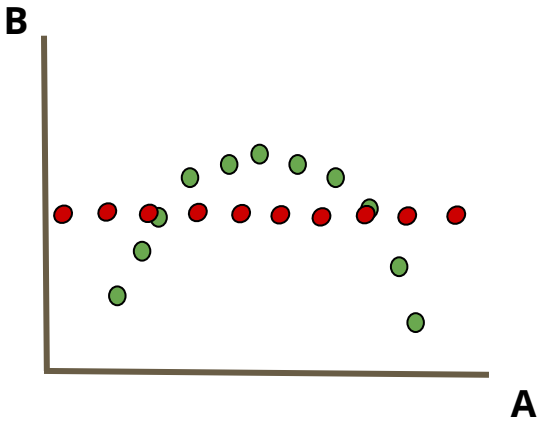
But we might be able to approximate the dataset with a lower rank one that contains similar information.

In Practice

But we might be able to approximate the dataset with a lower rank one that contains similar information.







Matrix Factorization Improved

Not only can we factorize a matrix **A** of rank **k** as **A = UV**. But we can factorize **A** using a process called Singular Value Decomposition where:

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathbf{T}}$$

Approximation

Definition:

The **Singular Value Decomposition** of a rank-r matrix A has the form

$$\mathbf{A} = \mathbf{U\Sigma V^T}$$

where

U is **n x r**
The **columns** of **U** are orthogonal & unit length ($\mathbf{U^TU = I}$)

V is **m x r**
The **columns** of **V** are orthogonal & unit length ($\mathbf{V^TV = I}$)

Approximation

Definition:

The **Singular Value Decomposition** of a rank-r matrix A has the form

$$\mathbf{A = U\Sigma V^T}$$

where

$$\Sigma = \begin{pmatrix} \sigma_1 & & & 0 \\ & \sigma_2 & & \\ & & \ddots & \\ 0 & & & \sigma_r \end{pmatrix}$$

with $\boldsymbol{\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r > 0}$

$\boldsymbol{\sigma_i}$ is the square root of the eigenvalues of $\mathbf{A^TA}$ and are called **singular values**

Approximation

Goal:

Approximate **A** with $\mathbf{A}^{(k)}$ (low-rank matrix) such that

1. $\mathbf{d(A, A^{(k)})}$ is small
2. **k** is small compared to **m** & **n**

Frobenius Distance

$$d_F(A, B) = \|A - B\|_F = \sqrt{\sum_{i,j} (a_{ij} - b_{ij})^2}$$

i.e. the pairwise sum of squares difference in values of A and B

Approximation

Definition:

When **k < rank(A)**, the **rank-k approximation** of **A** (in the least squares sense) is

$$A^{(k)} = \underset{\{B|rank(B)=k\}}{\arg\min} \; d_F(A, B)$$

Approximation

Find $\mathbf{A}^{(k)}$ by decomposing $\mathbf{A}$:

$$A = \begin{pmatrix} \boxed{U_1} & U_2 \end{pmatrix} \begin{pmatrix} \boxed{\Sigma_1} & \\ & \Sigma_2 \end{pmatrix} \begin{pmatrix} \boxed{V_1} & V_2 \end{pmatrix}$$

$$\mathbf{A}^{(k)} = \mathbf{U}_1 \mathbf{\Sigma}_1 \mathbf{V}_1^{\mathbf{T}}$$

Where

$\mathbf{U}_1$ is **n x k**
$\mathbf{\Sigma}_1$ is **k x k**
$\mathbf{V}_1$ is **m x k**

Approximation

1	1	1	0	0
2	2	2	0	0
1	1	1	0	0
5	5	5	0	0
0	0	0	2	2
0	0	0	3	3
0	0	0	1	1

=

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

9.64	0
0	5.29

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Approximation

1	1	1	0	0
2	2	2	0	0
1	1	1	0	0
5	5	5	0	0
0	0	0	2	2
0	0	0	3	3
0	0	0	1	1

~

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

9.64	0
0	0

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Approximation

1	1	1	0	0
2	2	2	0	0
1	1	1	0	0
5	5	5	0	0
0	0	0	2	2
0	0	0	3	3
0	0	0	1	1

~

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

9.64	0
0	0

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Approximation

1	1	1	0	0
2	2	2	0	0
1	1	1	0	0
5	5	5	0	0
0	0	0	2	2
0	0	0	3	3
0	0	0	1	1

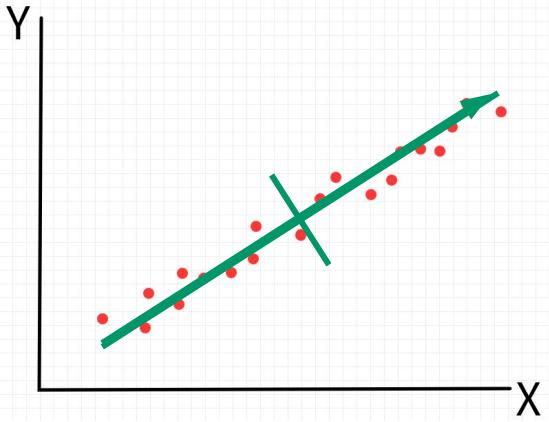
~

1	1	1	0	0
2	2	2	0	0
1	1	1	0	0
5	5	5	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Approximation

The **ith singular vector** represents the direction of the ith most variance.

$$\Sigma = \begin{pmatrix} \sigma_1 & & & 0 \\ & \sigma_2 & & \\ & & \ddots & \\ 0 & & & \sigma_r \end{pmatrix}$$



Singular Values express the importance / significance of a singular vector

Approximation

Property:

$$d_F(A, A^{(k)})^2 = \sum_{i=k+1}^{r} \sigma_i^2$$

Note: the larger **k** is, the smaller the distance.

Approximation

To find the right **k** you can:

1. Look at the singular value plot to find the elbow point
2. Look at the residual error of choosing different **k**

Related to Principal Component Analysis (PCA)

SVD and PCA are related

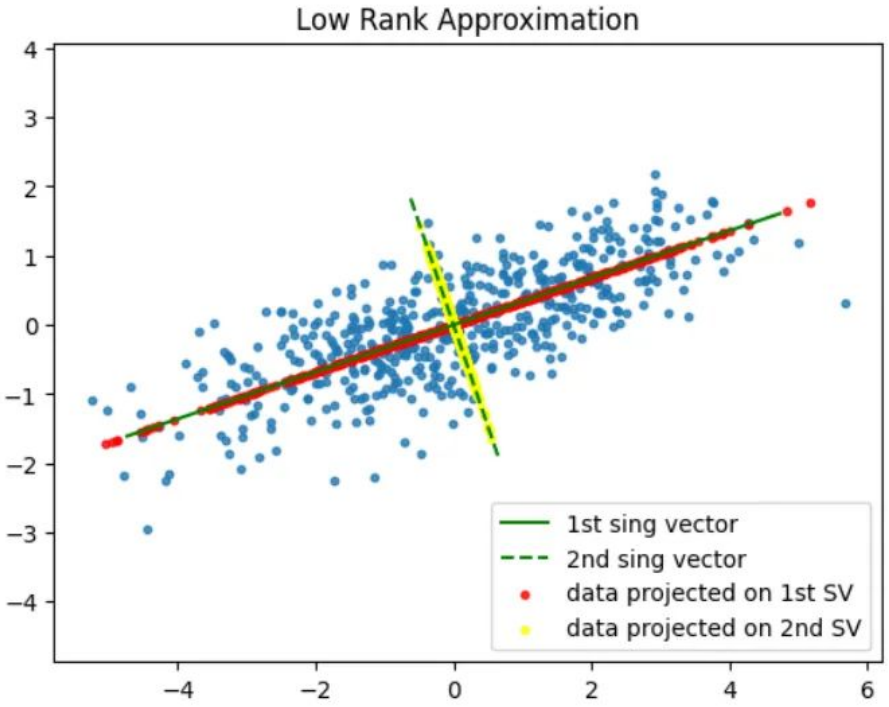
See demo

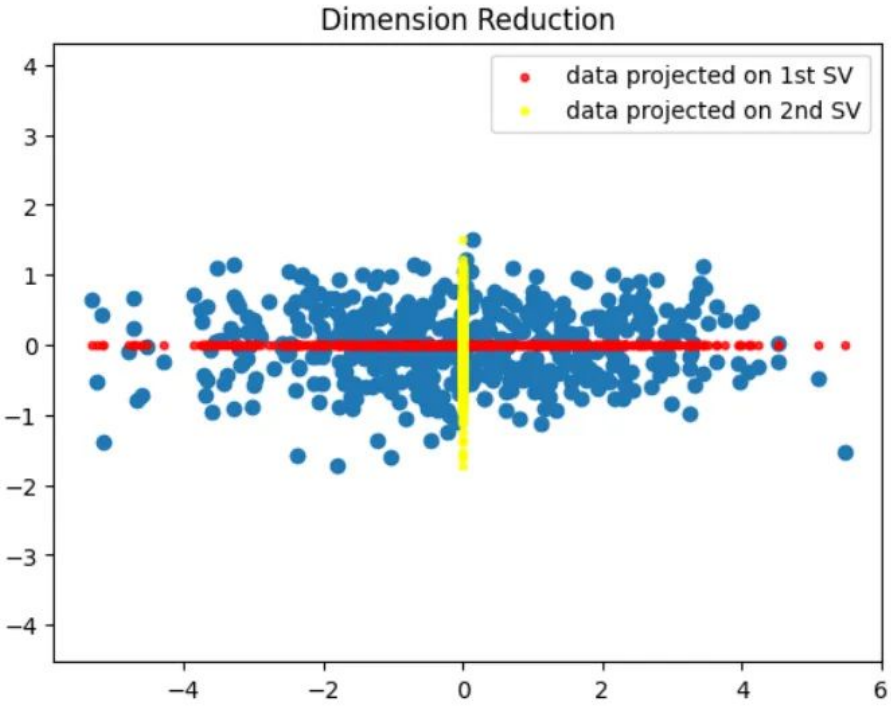
Dimensionality Reduction

Idea: project the data onto a subspace generated from a subset of singular vectors / principal components.

We want to project onto the components that capture most of the variance / information in the data.

Dimensionality Reduction VS Low Rank Approx.





Anomaly Detection

Define $\mathbf{O} = \mathbf{A} - \mathbf{A}^{(k)}$

The largest rows of **O** could be considered anomalies

Latent Semantic Analysis

Boston University CS 506 - Lance Galletti

Latent Semantic Analysis

	data	information	retrieval	brain	lung
CS-paper-1	1	1	1	0	0
CS-paper-2	2	2	2	0	0
CS-paper-3	1	1	1	0	0
CS-paper-4	5	5	5	0	0
Med-paper-1	0	0	0	2	2
Med-paper-2	0	0	0	3	3
Med-paper-3	0	0	0	1	1

Latent Semantic Analysis

1	1	1	0	0
2	2	2	0	0
1	1	1	0	0
5	5	5	0	0
0	0	0	2	2
0	0	0	3	3
0	0	0	1	1

=

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

9.64	0
0	5.29

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

1	1	1	0	0
2	2	2	0	0
1	1	1	0	0
5	5	5	0	0
0	0	0	2	2
0	0	0	3	3
0	0	0	1	1

=

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

9.64	0
0	5.29

x

Document embedding

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

1	1	1	0	0
2	2	2	0	0
1	1	1	0	0
5	5	5	0	0
0	0	0	2	2
0	0	0	3	3
0	0	0	1	1

=

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

Word embedding

9.64	0
0	5.29

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

CS concept MD concept

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

9.64	0
0	5.29

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

CS concept

MD concept

doc-to-concept similarity

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

9.64	0
0	5.29

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

doc-to-concept
similarity matrix

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

9.64	0
0	5.29

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

doc-to-concept
similarity matrix

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

"strength" of the CS concept

9.64	0
0	5.29

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

doc-to-concept similarity matrix

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

"strength" of the each concept

9.64	0
0	5.29

x

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

doc-to-concept similarity matrix

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

"strength" of the each concept

9.64	0
0	5.29

x

term-to-concept similarity

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

Latent Semantic Analysis

doc-to-concept similarity matrix

0.18	0
0.36	0
0.18	0
0.90	0
0	0.53
0	0.80
0	0.27

x

“strength” of the each concept

9.64	0
0	5.29

x

term-to-concept similarity matrix

0.58	0.58	0.58	0	0
0	0	0	0.71	0.71

What affects the quality of the embeddings found?

Recall the guiding principle: words that appear together should share meaning.

Latent Semantic Analysis

We can better represent each document by:

- The presence of each word (0 / 1)

Latent Semantic Analysis

We can better represent each document by:

- The presence of each word (0 / 1)
- Count of the word (0, 1, ...)

Latent Semantic Analysis

We can better represent each document by:

- The presence of each word (0 / 1)
- Count of the word (0, 1, ...)
- Frequency of the word (n_i / Σn_i)
- TfiDf

tf · idf

Term frequency in the document

$$\log\left(\frac{\text{number of documents}}{\text{number of documents that contain the term}}\right)$$

Limitations

Concepts are orthogonal...

Concept A · Concept B = 0

Recap

- SVD rotates our data into a new space where patterns become independent directions (latent patterns in our data)
  - But these axes are just linear combinations of the original features
    - For example if we have weight and height, SVD might find the following features:
      - Weight + Height
      - Weight - Height
- Our data has many features but few strong patterns. High singular values correspond to strong patterns. We can select the top k patterns to represent our data:
  - Help models extract information
  - Speed up our algorithms with matrix factorization
- Sometimes the noise is what's interesting: anomaly detection
- With a matrix of documents as represented by words, SVD can extract latent patterns and help embed words and documents in a space where words that share semantic meaning are close